🧱 Full-Stack Web Development and Stacks

Fulllion Creative Works

Contents

1. What "Full Stack" Means
2. The Route In
3. What a Stack Is, and How to Choose
4. MEAN and MERN
5. The T3 Stack
6. Next.js as a Full-Stack Framework
7. Backend as a Service: Firebase and Supabase
8. Comparing the Stacks
9. Web Performance Across the Stack
10. Capstone Project Ideas
11. Your Portfolio and Career
12. Sources

---

1. What "Full Stack" Means

A **full-stack developer** can build every layer of a web app:

Layer	What it is	Examples
Front end	What runs in the browser	HTML, CSS, JavaScript, React, Angular
Back end	The server code: APIs, sign-in, business rules	Node.js, Python, Next.js server code
Database	Where the data lives	PostgreSQL, SQLite, MongoDB, Firestore
Infrastructure	Where it all runs, and how it gets there	Hosting, domains, HTTPS, deploying from GitHub

Nobody is an expert in every layer. "Full stack" usually means you're strong in one or two, and comfortable enough in the rest to build and ship a complete app on your own.

For a fun look at how deep it goes, watch Fireship's short Fullstack Development Iceberg (2021). In 40 seconds it drops from a framework like React or Angular, through TypeScript or Babel, JavaScript, the browser or Node.js that runs it, and the V8 engine that compiles it to machine code, down to the CPU's logic gates, transistors and, as a joke, physics and consciousness. Don't let the bottom of the iceberg put you off: you only need the top layers to build real things, and the Computer Science Fundamentals guide covers the layers below when you're curious.

---

2. The Route In

There are two good ways in:

1. **Learn the front end, then the back end.** Work through Learn Front-End Web Development, then Learn Back-End Web Development, building projects at each stage. Once you can do both, you're full stack.
2. **Or take Harvard's CS50 Web course.** CS50's Web Programming with Python and JavaScript (CS50W) covers HTML and CSS, Git, Python, Django, SQL, JavaScript, React, testing and CI/CD, and scalability and security, with projects each week and a final project. Start Here compares it with The Odin Project, freeCodeCamp and MDN.

About CS50W, as of November 2026:

- **The course is free** on Harvard's site, as OpenCourseWare. You need a free edX account to submit projects.
- **A free CS50 certificate** is available if you score at least 70% on each project, including the final project.
- **edX also offers a paid verified certificate** for the course, and the paid **Computer Science for Web Programming Professional Certificate** (HarvardX), which pairs CS50W with CS50's introduction to computer science. Check edX for the current price.

Either way, follow the same five steps from the other guides: learn a language, build small projects, learn its frameworks, build bigger projects with Git and GitHub, then build a portfolio and pursue your career. Learn in any order you like, but don't take on too much at once.

---

3. What a Stack Is, and How to Choose

A **stack** is the set of technologies an app is built with, one for each layer. Some popular combinations got acronyms, like MEAN and MERN. Others are named after a person or a framework.

How to choose, as a learner:

1. **Start from a language you already know.** If you know JavaScript or TypeScript, a JavaScript stack lets you use one language everywhere. If you learned Python, Django gives you a complete stack on its own.
2. **Look at job listings where you want to work.** Some areas and industries lean heavily on Angular, Java or .NET. Others are mostly React and Node.js.
3. **Prefer fewer moving parts.** One framework that handles front and back end, plus a managed database, is easier to finish and deploy than five separate pieces.
4. **Pick one and finish a project.** Switching stacks halfway through is the classic way to never finish.

---

4. MEAN and MERN

MEAN and **MERN** are all-JavaScript stacks, named after their four parts:

Letter	MEAN	MERN
M	MongoDB (document database)	MongoDB
E	Express (back-end framework)	Express
A / R	**Angular** (front end)	**React** (front end)
N	Node.js (runs the back end)	Node.js

There's also **MEVN**, with Vue in place of Angular or React.

Academind's MEAN tutorial: what's actually in it

Academind's Angular - Node - MongoDB & Express (MEAN) Tutorial for Beginners is a free, two-and-a-half-hour excerpt from Maximilian Schwarzmüller's paid MEAN course. Despite the title, **it doesn't get as far as Express or MongoDB.** The first 20 minutes or so explain how the four parts work together; the rest builds the **Angular front end** of a small posts app. The back end, sign-in, image upload and deployment are in the full course. That still makes it a thorough Angular introduction. Jump to:

- The MEAN stack and how the pieces connect (0:47), including why MongoDB stores documents rather than table rows
- What a single-page app is (13:08), and why Angular talks to a separate API behind the scenes
- Installing the Angular CLI and creating a project (21:28), then a tour of the project's files (29:44)
- Building the first components (40:34) and event binding (1:01:26)
- Adding Angular Material for styled components (1:15:50)
- Forms with `ngModel` and two-way binding (1:45:18)
- Sharing data with a service and dependency injection (2:06:08), then updating components with an RxJS Subject (2:20:37)

It's an older Angular. It organizes the app with an `NgModule`, where new projects now use standalone components, and loops with `*ngFor`, which Angular 20 deprecated in favor of the built-in `@for` block (available since Angular 17). Learn the ideas here (components, binding, services, observables), and take current syntax from angular.dev. For the Express half, Learn Back-End Web Development, Section 7 builds a small API.

Where they stand in 2026:

- **Every part is current and maintained.** Angular 22 came out in June 2026, Express is on version 5, and Node.js 26 is the Active LTS line. Angular is especially common in larger companies.
- **The tutorials are older than the tools.** Angular has changed a lot since Academind's video: new projects now use standalone components rather than `NgModule`s. Express 5 changed some route syntax and error handling (Learn Back-End Web Development, Section 4). Follow along for the concepts, and check angular.dev for current code.

- **MongoDB isn't the default it once was.** Many new projects use a SQL database such as PostgreSQL instead, because most app data has relationships. MongoDB is still a solid choice when your data really is document-shaped.
- **React's docs now recommend starting with a framework.** For a new React app, react.dev suggests a full-stack framework such as Next.js or React Router, rather than plain React plus a separate Express server. MERN is still a good way to *learn* how a front end and an API talk to each other, though.

---

5. The T3 Stack

The **T3 stack** is a TypeScript-everywhere stack, popularized by Theo (t3.gg). Its core is **Next.js** and **TypeScript**, usually with **Tailwind CSS**, plus optional pieces:

Piece	Does
tRPC	Calls server functions from the front end with full type safety, without writing a separate REST API
Prisma or **Drizzle**	Talks to a SQL database with typed code (an ORM)
NextAuth.js (Auth.js) or **Better Auth**	Sign-in
Tailwind CSS	Styling

Its starter, `create-t3-app`, lets you pick which pieces you want. Its motto is "typesafety isn't optional": if you rename a database column, TypeScript shows you every place in the front end that breaks.

Theo's T3 tutorial: what's actually in it

Theo's three-hour T3 Stack Tutorial - FROM 0 TO PROD FOR $0 (Next.js, tRPC, TypeScript...) (2023) builds and deploys **Chirp**, a Twitter clone where you can only post emojis. His recurring advice is to **deploy early and often**, so the app goes live in the first few minutes and every later push deploys automatically. It uses:

- `create-t3-app` with Prisma, Tailwind CSS and tRPC, but **Clerk** for sign-in rather than NextAuth.js, because he'd come to prefer a hosted sign-in service;
- **PlanetScale** (a hosted MySQL database), **Vercel** for hosting, **Axiom** for logs, and **Upstash** (hosted Redis) for rate limiting, so one user can't flood the site;
- **Zod** to check each tRPC input, and a "private procedure" that only signed-in users can call;
- Next.js's older **Pages Router**, with tRPC's helpers and `getStaticProps` to build profile and post pages ahead of time;
- a **GitHub Actions** workflow that runs the linter and the TypeScript type check on every push, and a custom domain at the end.

Jump to:

- Creating the project with create-t3-app (2:05)

- Setting up the database (6:13)
- Adding sign-in with Clerk (11:41)
- Logging in production (20:08)
- How tRPC works (24:55)
- Protected procedures for signed-in users (1:25:17)
- Rate limiting (1:38:19)
- Building pages ahead of time (2:06:16)
- Type checking and linting in GitHub Actions (2:47:40)

Watch it for the habits (deploying early, validating input, rate limiting, checking types in CI), not the exact setup. Three things have changed: new Next.js projects use the **App Router**, not the Pages Router it teaches; **PlanetScale's free plan is gone** (below), so the "$0" in the title no longer holds as filmed; and Clerk, Axiom and Upstash set their own free-plan limits, so check each one's pricing before you rely on it.

Where it stands in November 2026:

- `create-t3-app` **still works, but it has slowed down.** Its latest release is **7.40.0, from November 2025**, and the last change to its code was a security update in December 2025. It still sets up **Next.js 15**, while Next.js itself is on **16.4**.
- **Its choices have moved on.** It now offers **Drizzle** alongside Prisma, and **Better Auth** alongside NextAuth.js. The Auth.js project (which includes NextAuth.js) is now part of Better Auth.
- **Free hosting in the tutorial may have changed.** Free tiers come and go. For example, PlanetScale retired its free Hobby database plan in April 2024. If a service in the tutorial no longer has a free plan, swap in a current one.
- **Server Actions now cover much of what tRPC was for.** Next.js can call server code from forms and buttons with type safety built in (Section 6), so many new projects skip tRPC.

The ideas behind T3 (TypeScript end to end, a typed database layer, only adding tools that solve a real problem) are still excellent. If you use the starter, update Next.js afterwards, or start from `create-next-app` and add the pieces yourself.

---

6. Next.js as a Full-Stack Framework

Next.js (version 16.4, October 2026) is a React framework that handles the front end *and* the back end in one project. For many apps, it replaces a separate Express server. It has three main ways to run server code.

6.1 Server Components: reading data

Pages and components in the `app` folder run on the server by default, so they can read the database directly and send finished HTML to the browser. Your database password never reaches the browser. Learn Front-End Web Development, Section 11 explains where Server Components came from, and how the App Router uses them.

6.2 Server Actions: changing data from your own pages

A **Server Function** is an `async` function marked with `'use server'`. When it's used to handle a form or a change, it's called a **Server Action**. The browser calls it for you, behind the scenes, with a `POST` request.

```
// app/notes/actions.ts
'use server';

import { revalidatePath } from 'next/cache';
import { getCurrentUser } from '@/lib/auth'; // your own sign-in check
import { db } from '@/lib/db'; // your database client

export async function addNote(formData: FormData) {
  const user = await getCurrentUser();
  if (!user) throw new Error('Not signed in');

  const text = String(formData.get('text') ?? '').trim();
  if (!text) return;

  await db.note.create({ data: { text, userId: user.id } });
  revalidatePath('/notes'); // show the new note
}
```

```
// app/notes/page.tsx
import { addNote } from './actions';

export default function NotesPage() {
  return (
    <form action={addNote}>
      <input name="text" placeholder="New note" />
      <button type="submit">Add</button>
    </form>
  );
}
```

> **Check sign-in inside every Server Action.** The Next.js docs warn that Server Functions can be reached with a direct `POST` request, not only through your page. Treat each one like a public API route.

6.3 Route Handlers: APIs for everything else

A **Route Handler** is a `route.ts` file that exports a function for each HTTP method. Use one when something *other than your own pages* needs to call your server: a mobile app, another website, or a webhook from a service like a payment provider.

```
// app/api/notes/route.ts
import { getCurrentUser } from '@/lib/auth';
import { db } from '@/lib/db';

export async function GET() {
  const user = await getCurrentUser();
  if (!user) {
    return Response.json({ error: 'Not signed in' }, { status: 401 });
  }
  const notes = await db.note.findMany({ where: { userId: user.id } });
  return Response.json(notes);
}
```

This answers `GET /api/notes`. Supported methods are `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD` and `OPTIONS`.

6.4 Which to use

You want to	Use
Show data on a page	A Server Component
Handle a form or button in your own app	A Server Action
Offer an API to other apps, or receive webhooks	A Route Handler

The Private Cloud Storage guide shows real Route Handlers that sign file uploads and downloads.

State that lives only in the browser, such as form inputs and open menus, still belongs in Client Components. State Management in React Today covers built-in state, context, TanStack Query and Redux Toolkit.

---

7. Backend as a Service: Firebase and Supabase

A **backend as a service (BaaS)** gives you a ready-made back end: a database, sign-in, file storage and hosting, managed for you. You write the front end and the rules, and skip running servers. It's the fastest way to ship a real app, and a very common choice for portfolio projects and startups.

	Firebase (Google)	**Supabase**
Database	Cloud Firestore, a NoSQL document database with real-time sync and offline support. Data Connect adds PostgreSQL (through Cloud SQL)	PostgreSQL, with auto-generated REST and GraphQL APIs
Sign-in	Firebase Authentication	Supabase Auth
Files	Cloud Storage	Supabase Storage
Server code	Cloud Functions, and App Hosting for Next.js and Angular apps	Edge Functions
Real-time	Built into Firestore	Realtime subscriptions

	Firebase (Google)	**Supabase**
Open source	No	Yes (Apache 2.0), and can be self-hosted
Security	Security Rules decide who can read and write what	Postgres row-level security does the same job

Watch out for:

- **The rules are your security.** With a BaaS, the browser often talks to the database directly, so your Security Rules (Firebase) or row-level security policies (Supabase) are the only thing stopping one user reading another's data. Test them.
- **Costs grow with use.** Firebase App Hosting and Cloud Storage need the pay-as-you-go Blaze plan. Set a budget alert.
- **Lock-in.** Moving off a BaaS later is real work. Supabase is plain PostgreSQL underneath, which makes leaving easier.

A real example

These three guides follow one real Next.js and Firebase app from start to finish:

1. Building Firebase Apps in Google Antigravity IDE: setting up Next.js and Firebase, the Firebase CLI, local emulators, Firestore Security Rules and their tests, and continuous deployment.
2. Connecting a Custom Domain to Firebase App Hosting: putting the app online with your own domain, publishing to GitHub, and deploying changes.
3. Private Cloud Storage for Firebase App Hosting: adding private file storage with signed upload and download links.

---

8. Comparing the Stacks

Stack	Front end	Back end	Database	Language	Best for	Watch out for
MEAN	Angular	Express on Node.js	MongoDB	TypeScript and JavaScript	Learning; jobs in companies that use Angular	Older tutorials; a separate front end and API to deploy
MERN	React	Express on Node.js	MongoDB	JavaScript or TypeScript	Learning how a front end and an API talk	React now recommends a framework for new apps
T3	Next.js and React	Next.js, with tRPC	SQL, via Prisma or Drizzle	TypeScript	Type-safe apps; TypeScript fans	Starter trails current Next.js (Section 5)

Stack	Front end	Back end	Database	Language	Best for	Watch out for
Next.js full stack	Next.js and React	Server Actions and Route Handlers	Any (often PostgreSQL)	TypeScript or JavaScript	Most new React apps	You choose and wire up the database and sign-in yourself
Firebase	Any (often Next.js)	Firebase services, Cloud Functions, App Hosting	Firestore (NoSQL), or PostgreSQL via Data Connect	Any	Shipping fast; real-time apps; portfolio projects	Rules are your security; pay-as-you-go costs; lock-in
Supabase	Any (often Next.js)	Supabase services, Edge Functions	PostgreSQL	Any	Shipping fast, on SQL	Row-level security must be set up right
Django	Django templates, or any front end	Django	PostgreSQL or SQLite	Python	Python learners (CS50W uses it); content and admin-heavy sites	Less common in JavaScript-focused job listings

If you have no strong reason to choose otherwise, **Next.js plus PostgreSQL** (yourself or through Supabase) or **Next.js plus Firebase** covers a huge range of projects and job listings in 2026.

---

9. Web Performance Across the Stack

Speed comes from every layer: how much the front end sends, how fast the back end answers, and how well everything is cached. For an overview, watch Beyond Fireship's The ultimate guide to web performance (2023), which covers LCP, CLS and responsiveness, and tests big sites with Unlighthouse. Two parts are dated: it uses First Input Delay, which INP has replaced, and the Web Vitals extension it demonstrates is no longer supported, since its features moved into Chrome DevTools. The front-end guide's performance section summarizes the video and the current tools.

9.1 Core Web Vitals

Google's **Core Web Vitals** measure what users actually feel. A page passes each one when at least 75% of visits meet the "good" mark:

Metric	Measures	Good
Largest Contentful Paint (LCP)	How fast the main content shows up	2.5 seconds or less
Interaction to Next Paint (INP)	How fast the page responds to clicks and typing	200 milliseconds or less
Cumulative Layout Shift (CLS)	How much the page jumps around while loading	0.1 or less

INP replaced the older First Input Delay (FID) metric in 2024, so ignore tutorials that still talk about FID.

To measure them, use PageSpeed Insights for a live site, and the **Performance** panel in Chrome DevTools while you work.

9.2 What each layer can do

Layer	Helps with	What to do
Front end	LCP, CLS, INP	Send correctly sized, compressed images, and never lazy-load the main one. Give images a width and height so the page doesn't jump. Ship less JavaScript, and break up long tasks. Learn Front-End Web Development covers images in more depth
Back end	LCP (through time to first byte)	Make slow database queries faster with indexes, and avoid running one query per item in a list. Render pages on the server or ahead of time. Host the server close to its database
Network and hosting	LCP	Use a CDN, keep redirects to a minimum, and cache (below)

9.3 Caching basics

The fastest request is the one you don't make. HTTP caching is controlled by the `Cache-Control` header your server sends:

Content	Header	Why
Files with a version or hash in the name, such as `app.3f9a1c.js`	`Cache-Control: public, max-age=31536000, immutable`	The name changes whenever the content does, so browsers and CDNs can keep it for a year
HTML pages	`Cache-Control: no-cache`, with an `ETag`	Browsers keep a copy, but check with the server first. If nothing changed, the server answers `304 Not Modified` with no body
Personal data, such as a user's account page	`Cache-Control: private` (or `no-store` for sensitive pages)	Only the user's own browser may keep it, never a shared CDN

Beyond HTTP caching, back ends cache expensive results in memory or in a store such as Redis, and frameworks add their own layers. Next.js, for example, can cache data with `'use cache'`. Whatever you cache, know how it gets refreshed when the data changes.

---

10. Capstone Project Ideas

A **capstone** is the bigger project that pulls everything together, and the centerpiece of your portfolio. Pick one idea and build it all the way: deployed, with sign-in, real data and a good README.

Project	What it shows
Notes or task app with accounts	Create, read, update and delete (CRUD), sign-in, and making sure users only see their own data
Recipe, bookmark or reading tracker	Database relationships, tags, search and pagination
Event sign-up or booking app	Forms, validation, time zones, and confirmation emails
Budget or expense tracker	Totals and grouping in SQL, charts, and CSV export
Q&A board or small social feed	Likes, follows, pagination, and moderation

Project	What it shows
Real-time chat or shared whiteboard	Real-time updates through Firestore, Supabase Realtime or WebSockets
Small shop using a payment provider's test mode	A cart, checkout, and webhooks through a Route Handler, without any real payments
File-sharing or photo portfolio app	Uploads, private storage and signed links, as in Private Cloud Storage for Firebase App Hosting
URL shortener with click stats	Redirects, caching, rate limiting and simple analytics
Dashboard for a public dataset or API	Calling external APIs, caching their results, and presenting data clearly
Job application tracker	A practical CRUD app you'll actually use while job hunting

What makes a capstone stand out:

1. **It's live,** at a real address, ideally your own domain.
2. **It has a clear README** with screenshots, the stack, how to run it, and what you'd do next.
3. **It handles the boring parts:** loading and error states, empty states, validation, and working on a phone.
4. **It has tests** for the important parts, run automatically on each push.
5. **It's secure:** no secrets in the repository, and permissions checked on the server.

For smaller practice projects to build first, see Coding Practice and Interview Prep. For the data structures and algorithms that make features like search fast, see Computer Science Fundamentals.

---

11. Your Portfolio and Career

Once you have a capstone or two, it's time for step 5: a portfolio, and the job hunt. Web Developer Careers, Portfolio and Freelancing covers portfolio sites, GitHub profiles, résumés, job hunting, and freelancing, including freeCodeCamp's freelancing guide. Its Projects That Show Real Skill section lists what makes a project stand out to employers. Interview preparation is in Coding Practice and Interview Prep.

---

12. Sources

Courses and roadmaps

- CS50's Web Programming with Python and JavaScript and CS50W certificate
- edX: Computer Science for Web Programming Professional Certificate
- roadmap.sh: Full Stack Developer

Stacks and frameworks

- MongoDB: What is the MERN stack? and What is the MEAN stack?
- Angular release and support policy, Angular: Control flow and NgFor (deprecated since v20)

- Express and Node.js release schedule
- React versions and Creating a React App
- Create T3 App: Introduction and create-t3-app on GitHub
- tRPC, Prisma, Drizzle ORM, Better Auth and Auth.js
- PlanetScale: Hobby plan deprecation FAQ
- Next.js blog (release history)
- Next.js: Mutating Data (Server Functions and Server Actions)
- Next.js: Route Handlers and Backend for Frontend

Backend as a service

- Firebase documentation, Cloud Firestore, Data Connect and App Hosting
- Supabase documentation and Supabase on GitHub

Performance and caching

- web.dev: Web Vitals and Optimize Largest Contentful Paint
- PageSpeed Insights
- Chrome DevTools: Performance panel and Support for the Web Vitals extension has ended
- MDN: HTTP caching

Videos

- Fireship, Fullstack Development Iceberg
- Academind, Angular - Node - MongoDB & Express (MEAN) Tutorial for Beginners
- Theo, T3 Stack Tutorial - FROM 0 TO PROD FOR $0 (Next.js, tRPC, TypeScript...)
- Beyond Fireship, The ultimate guide to web performance

Latest version: github.com/ImmuneMoon/Full-Stack-Web-Development-and-Stacks

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.